

Docker unit 1 overview:

1. DevOps

DevOps stands for Development and Operations. It is a practice that focuses on improving collaboration between development and operations teams. It helps in faster software delivery, automation of processes, and reliable deployment.

Main goals:

- Faster development and deployment
- Continuous integration and delivery
- Improved collaboration

2. Problem with Traditional Systems

Earlier, one application was deployed on one physical server. This led to:

- Wastage of resources
- High cost
- Poor scalability

This problem led to the use of virtualization and containerization.

3. Virtualization

Virtualization is the process of creating multiple virtual machines (VMs) on a single physical server using a hypervisor.

Each VM has:

- Its own operating system
- Its own applications

Advantages:

- Better hardware utilization
- Strong isolation
- Can run multiple operating systems

Disadvantages:

- Heavy resource usage
- Slow boot time
- High maintenance cost

4. Containerization

Containerization is a lightweight method of running applications in isolated environments called containers.

Containers:

- Share the host operating system
- Do not require a full OS

Advantages:

- Fast startup
- Lightweight
- Highly scalable
- Portable across systems

Containers are mainly used in microservices architecture.

5. Container Runtime

A container runtime is software that runs containers. It converts a container image into a running container.

Types:

- High-level runtimes: Docker, containerd, CRI-O
- Low-level runtime: runc

6. Namespaces

Namespaces are used for isolation in containers. They ensure that each container works independently.

Types of namespaces:

- PID: isolates processes
- Network: provides separate IP and ports
- Mount: isolates file system
- UTS: provides hostname
- User: improves security

Important point:

Namespaces provide isolation between containers.

7. cgroups (Control Groups)

cgroups are used to control and limit resource usage of containers.

They manage:

- CPU usage
- Memory usage
- Disk I/O

Importance:

- Prevents one container from using all resources
- Ensures fair usage

Important point:

Namespaces isolate, cgroups control.

8. Container Image

A container image is a read-only package that contains:

- Minimal OS
- Application code
- Dependencies

It acts as a blueprint for containers.

Properties:

- Immutable
- Reusable

9. Image Layers

Docker images are made up of multiple layers. Each instruction in a Dockerfile creates a new layer.

Benefits:

- Faster builds using caching
- Reduced storage
- Faster deployment

10. Image Registry

An image registry is a storage system used to store and distribute container images.

Types:

- Public registry (e.g., Docker Hub)
- Private registry (used in enterprises)

Importance:

- Version control of images
- Easy sharing
- Used in CI/CD pipelines

11. CI/CD with Docker

Process:

12. Code is pushed to repository
13. Image is built using CI
14. Image is stored in registry
15. Image is deployed using CD

The registry acts as a bridge between CI and CD.

12. Docker Architecture

Main components:

- Docker Client
- Docker Daemon
- Docker Images
- Containers
- Registry

13. Docker Lifecycle

Stages:

- Build: create image
- Ship: push image to registry
- Run: execute container

14. Basic Docker Commands

Pull image:

```
docker pull ubuntu
```

List images:

```
docker images
```

Run container:

```
docker run ubuntu
```

Stop container:

```
docker stop <container_id>
```

Remove container:

```
docker rm <container_id>
```

Remove image:

```
docker rmi ubuntu
```

15. Interactive and Detached Mode

Interactive mode:

```
docker run -it ubuntu /bin/bash
```

Detached mode:

```
docker run -d -p 8080:80 nginx
```

16. Debugging Commands

```
docker exec -it <container_id> bash
```

```
docker logs <container_id>
```

```
docker inspect <container_id>
```

```
docker stats
```

```
docker top <container_id>
```

17. Volumes

```
docker volume create myvolume
```

```
docker volume ls
```

```
docker volume inspect myvolume
```

```
docker volume rm myvolume
```

Used for persistent storage.

18. Networking

```
docker network create mynetwork
```

```
docker network ls
```

```
docker network inspect mynetwork
```

Used to connect containers.

19. Cleanup Commands

```
docker system prune -a  
docker stop $(docker ps -aq)  
docker rm $(docker ps -aq)
```

20. Key Differences

Virtualization vs Containerization:

- Virtualization uses full OS, containers share OS
- Virtualization is heavy, containers are lightweight
- Virtualization is slow, containers are fast
- Virtualization consumes more resources, containers use fewer